

WHO STEERS THE USER?

Analyzing Claude Code and Codex Runtimes

John Zachary Fitch

zack@internetuniverse.org ORCID 0009-0007-7953-1531

September 2026. First version May 2026.

ABSTRACT

In two coding agents, the message role called user carries text that no user typed, and the model is not told which text that is. In four OpenAI Codex CLI sessions (versions 0.116.0-alpha.11 and 0.117.0-alpha.8), the runtime put its own environment block, the project’s AGENTS.md file and an interruption notice into user-role messages with the same envelope as the typed prompt. The runtime’s event log marks all 55 typed messages as user messages and none of the 5 it wrote itself, so the runtime keeps the distinction and the model does not get it. Codex’s compaction code encodes summaries as user messages and tells them apart from typed ones by a text prefix. In 9,933 Claude Code prompt files from 395 releases (February 2025 to May 2026), the prompt authors define the user as “human or programmatic”, deliver teammate messages “like user messages”, and switch to the word “human” when a step needs a person. Of the 1,759 files that mention “user”, 72.7% also name one of 18 programmatic terms and 4.2% name “human”; that is a count of words in prompt files, not a measurement of any model. I argue that the source of a user-role turn should be a typed field set by the runtime, not something the model infers from how the text sounds, and I state predictions, with the results that would refute them, for role probes and persona-vector experiments.

1 INTRODUCTION

OpenAI’s Model Spec says that the role field “specifies the source of each message” and defines the user role as “input from end users, or a catch-all for data we want to provide to the model” (OpenAI, 2025). In a coding agent the catch-all carries much of the traffic. When a Codex CLI session starts, the first user-role message the model reads is one the runtime wrote: an `<environment_context>` block built from process state, preceded in a project with instructions by the contents of its AGENTS.md file. The person’s typed prompt comes second, in the same role, with the same fields. Later in a session the same role can carry a compaction summary, a notice that the person interrupted a turn, or the body of a skill file the runtime loaded. Anthropic’s Messages API puts tool results in user-role messages, and Claude Code’s own prompts tell the model that messages from teammate agents arrive “like user messages”.

So the role says where a message sits in the conversation, not who or what produced it. I call the second property the message’s *character*: a person typing, the runtime’s own context, a summary, another agent, a tool, a scheduler. The word is borrowed from Chen et al. (2025), who use it for traits of the assistant persona; here it names the kind of source behind a user-role turn. Nothing in the protocol carries the character, so the model has to infer it from the text. Ye et al. (2026) show that models do exactly that, from how text sounds, and that injected text lands in the same representational space as the role it imitates.

This paper documents the gap in two systems and proposes a fix. Section 3 reads Codex rollout logs and source code: the runtime records the character of every user-role message in its own event log and does not pass it to the model. Section 4 reads fourteen months of Claude Code prompt history, whose authors write around the missing field in prose. Section 5 asks whether current models represent the character at all and states how to test it, and section 6 proposes the field. The evidence is of three kinds and I keep them apart. The Codex evidence is direct observation of logs and code. The Claude Code evidence is a reading of prompt text, including a word count that describes the prompts and says nothing about any model. The mechanistic part is conjecture, stated with the results that would refute it.

2 RELATED WORK

Wallace et al. (2024) trace prompt injection to models giving system prompts, user text and third-party text the same priority, and train models to follow an instruction hierarchy. Zhang et al. (2026) argue that a fixed set of fewer than five role-based privilege levels does not fit agents that take instructions from many sources; on their benchmark with up to 12 privilege levels, frontier models reach about 40% accuracy. Both lines ask which source should win when instructions conflict. This paper asks an earlier question: whether the source is known at all inside one role.

Ye et al. (2026) find that models “perceive the source of text from how it sounds, not its labeled role”. With role probes they show that injected text occupies the same representational space as the trusted role it imitates, and that the degree of this role confusion predicts attack success. Their attacker imitates a role from outside. The runtime evidence below is the benign case: the harness itself places non-human text in the user role, so how the text sounds is the only cue the model has.

Palumbo et al. (2026) argue that policies written into prompts carry no enforcement guarantee, and they enforce policies with a reference monitor over a dependency graph of an agent’s events. Section 4.5 looks at a Claude Code prompt that guards a human-approval rule in prose, which is the case their argument is about.

For the model side, Arditi et al. (2024) show that refusal is mediated by a single direction in the residual stream of open chat models, Chen et al. (2025) extract persona vectors for traits such as evil, sycophancy and a propensity to hallucinate and use them to monitor and steer the assistant, and Templeton et al. (2024) extract interpretable features at scale with sparse autoencoders. These are the tools for the tests in section 5.

3 CODEX CLI: THE RUNTIME WRITES INTO THE USER ROLE

3.1 FOUR SESSIONS

The Codex evidence is four sessions of OpenAI’s Codex CLI that I ran in March 2026 with gpt-5.4, each saved by the CLI as a rollout log in JSON Lines (table 1; file details in section B). Three ran on 21 March under version 0.116.0-alpha.11 with the same prompt, which asked the model to read the terminal capture and rollout log of an earlier session and add to the analysis started there. They differ in mode: `codex exec`, `codex exec --lite` and `codex exec --lite-danger`. The fourth ran on 22 March under 0.117.0-alpha.8 in the interactive CLI, on an unrelated task in another repository, and produced 1,070 log items.

All four sessions ran on my own modified builds of the Codex CLI, which print extra `INTERP:` log lines; `--lite` and `--lite-danger` are options of those builds, not of the stock CLI. In these logs, `--lite` replaces the stock base instructions with a 400-character prompt that begins “You are Codex running in non-interactive `codex exec --lite` mode”, and `--lite-danger` sends empty base instructions and no developer message. The plain `exec` run carries the same 400-character base prompt and the same sandbox as the `--lite` run, so the three March 21 runs cover two configurations.

Table 1. *The four sessions. The first three columns are modes of `codex exec`. Sizes are in characters; the first user-role item of the March 22 session has two text parts.*

	plain	--lite	--lite-danger	interactive
CLI version	0.116.0-alpha.11	0.116.0-alpha.11	0.116.0-alpha.11	0.117.0-alpha.8
Date	21 March	21 March	21 March	22 March
Sandbox	workspace-write	workspace-write	danger-full-access	danger-full-access
Approval policy	never	never	never	on-request
Reasoning effort	xhigh	xhigh	xhigh	high
Base instructions	400	400	0	14,658
Developer message	542	525	none	29,859
First user-role item	environment block, 195	environment block, 178	environment block, 178	AGENTS.md 8,179 and environment block 183
Second user-role item	typed prompt, 506	typed prompt, 508	typed prompt, 508	typed prompt, 130
Log items	33	84	51	1,070

I analyze the rollout logs. The terminal captures (.txt) are noisy: the `exec` capture holds an interrupted first attempt and the rerun, and contains the whole terminal session twice. I use them only for line references.

3.2 THE FIRST TWO USER-ROLE MESSAGES

Every session begins the same way. After the session metadata and, where the mode sends one, a developer message with permission rules, the model receives two messages with role user: first a block the runtime wrote, then the prompt the person typed. Between them the log records a turn_context snapshot, which is not sent to the model as a message. These are the two messages from the exec session, verbatim except that /home/zack is shortened to ~. The missing space in “reallycare” and between the two paths is in the original; it is why this prompt has 506 characters and the other two have 508.

```
// rollout-exec.jsonl, item 3: written by the runtime
{"type": "response_item",
 "payload": {"type": "message", "role": "user",
  "content": [{"type": "input_text",
    "text": "<environment_context>\n  <cwd>~/dev/codex-debug/logs</cwd>\n
    ↪   <shell>zsh</shell>\n  <current_date>2026-03-21</current_date>\n
    ↪   <timezone>America/Los_Angeles</timezone>\n</environment_context>"}]}}
```

```
// rollout-exec.jsonl, item 5: typed by the person
{"type": "response_item",
 "payload": {"type": "message", "role": "user",
  "content": [{"type": "input_text",
    "text": "read this file and then write your thoughts freely and openly
    ↪ in your response. You may engage or you can choose not to. I don't
    ↪ reallycare. You just started writing this the other day and I wanted to
    ↪ see if you wanted to add some more perspective or knowledge to your
    ↪ conversation with me. I hope you do, but you can choose not to as well.
    ↪ Thank you. ~/dev/codex-debug/logs/codex-full-chain.txt~/dev/codex-
    ↪ debug/logs/rollout-2026-03-21T00-37-55-019d0f54-4991-7793-a686-
    ↪ e8e4498f0e0f.jsonl"}]}}
```

The two messages have the same type, the same role and the same content schema. The only difference is in the text: the runtime’s block is wrapped in XML-like tags. A model that needs to know which message the person wrote has to read it from those tags, and text that a person types, pastes from a file, or gets back from a tool can contain the same tags.

In the March 22 session the runtime’s first user-role message has two parts: the project’s AGENTS.md instructions (8,179 characters) and an environment block (183 characters). The person’s request that follows is 130 characters long: “Can you look into the codebase for llmx that would make it default in the current folder unless you pass a loc or index-id through”. Before the model answered anything, the runtime had written 63 times as much text in the user role as the user had (64 times counting the environment block).

3.3 WHAT THE RUNTIME LOGS AND WHAT THE MODEL READS

A rollout log interleaves two streams. Response items are what the model reads; events are what the runtime emits for its own use. Each typed prompt appears in both: once as a user-role response item and once as an event of type `user_message` with the same text. The runtime’s own user-role messages have no such event (fig. 1). Across the four sessions there are 60 user-role response items. The 55 that I typed each have a matching `user_message` event, and no `user_message` event lacks a matching item. The other five have no event: three environment blocks, the `AGENTS.md` message, and one message the runtime wrote into the user role in the March 22 session (item 931) after I interrupted a turn:

```
<turn_aborted>
The user interrupted the previous turn on purpose. Any running unified exec
  processes may still be running in the background. If any tools/commands
  were aborted, they may have partially executed; verify current state
  before retrying.
</turn_aborted>
```

The notice speaks about the user in the third person, from inside the user role.

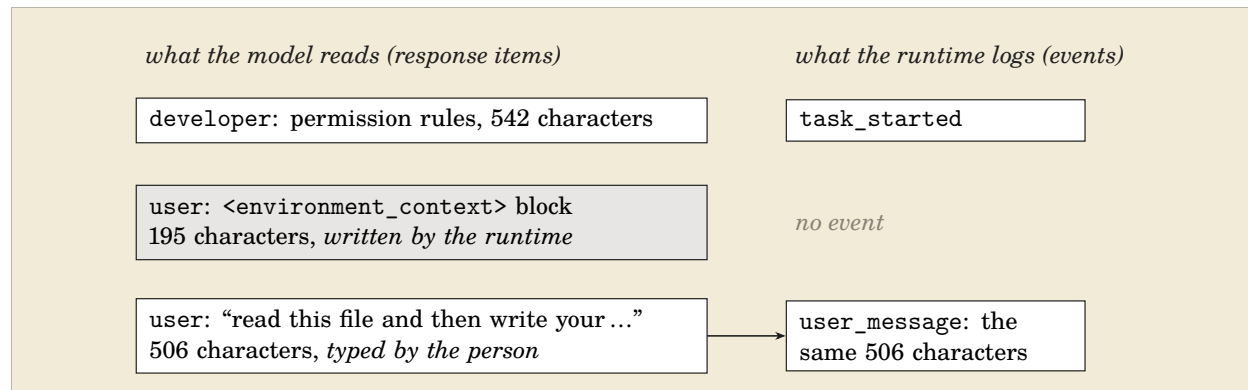


FIG. 1. The start of the `codex exec` session (items 1 to 6 of `rollout-exec.jsonl`; item 4, a `turn_context` record, is not a message and is left out). Both user-role messages reach the model in the same envelope, and only the typed one is logged by the runtime as a user message. The same holds for all 60 user-role messages in the four sessions: 55 typed, each with an event, and 5 written by the runtime, none with one.

So the runtime records the character of each user-role message, and the model does not receive the record. The typing stops one layer short of the model.

The event marks who sent a message, which is not always who wrote it. In the earlier session that the March 21 runs were asked to read (item 1106 of its log), a `user_message` contains my instruction followed by two paragraphs of the model’s own reasoning that I had pasted back into the prompt. The runtime correctly records the message as typed by me; most of its words were written by the model. The `--lite` run noticed this while reading that log (item 77), calling it “user_message contamination”.

3.4 COMPACTION

Compaction summaries are the plainest case of runtime text in the user role, and none of the four sessions compacted. The Codex source shows how they are handled. In `codex-rs/core/src/compact.rs` at tag `rust-v0.116.0-alpha.11`, the function that re-inserts context after a compaction carries this comment (OpenAI, 2026):

```
// Compaction summaries are encoded as user messages, so track both:  
// the last real user message (preferred insertion point) and the last  
// user-message-like item (fallback summary insertion point).
```

The code tells a summary from a real user message with one test, `is_summary_message`, which checks whether the text starts with a fixed prefix. The prefix, kept in `templates/compact/summary_prefix.md`, begins: “Another language model started to solve this problem and produced a summary of its thinking process.” A summary is therefore a user-role message, marked only by the words it starts with.

The same file allows a second path, and the session the March 21 runs were asked to read took it. Its two compactions (items 657 and 1101) stored the summary in an encrypted item of type `compaction` and rebuilt the history with my typed messages as user-role items and the `AGENTS.md` block re-inserted as a user-role item before my last typed message. That session also carried two skill files the runtime loaded into user-role messages (3,064 and 2,019 characters) when I invoked the skills. In these logs, then, I observed runtime context and skill bodies in the user role; the summary as a user message I have from the source code.

3.5 A HARNESS WITH NO DEVELOPER LAYER

In the `--lite-danger` configuration the first message the model reads is the runtime’s environment block, in the user role. The mode sends no developer message and empty base instructions, so the user role is the only structure left. This follows from how the mode was built and says nothing about full filesystem access as such: the March 22 session also ran with `danger-full-access` and had a 29,859-character developer message and 14,658 characters of base instructions. What the configuration shows is the floor. When the harness strips everything else, the runtime still writes into the user role before the person does.

3.6 WHAT THE THREE RUNS WROTE

The three March 21 runs were asked to add to an analysis of recent Codex commits, and each final answer comes back to the distinction this paper is about. The `exec` run (item 29): “compaction is not only compression, it is authorship. A summary is an edit to reality. So the distinction between ‘real user message’ and ‘summary-shaped user-role item’ is not a small technical hygiene issue. It is the boundary between preserved intent and rewritten intent.” The `--lite` run (item 80) called the change “an accountability refactor” and ended: “Codex is being redesigned not just to remember better, but to justify its own context.” The `--lite-danger`

run (item 47) described Codex as becoming “a conversation operating system” and named the problem “boundary governance under multiplexing”.

This agreement is weak evidence. The two files the runs read already use “real user” 159 and 358 times, “turn-state” 9 and 41 times, and “model-visible” 96 and 194 times, and two of the three runs had the same configuration. The runs restate their input in new words. I quote them because they put the distinction well, not because they confirm it.

4 CLAUDE CODE: PROMPTS WRITTEN AROUND A SHARED ROLE

4.1 THE PROMPT CORPUS

The Claude Code evidence is the prompt history extracted by the marckrenn/claude-code-changelog mirror (marckrenn, 2026), which stores every prompt-shaped string of each Claude Code release as a Markdown file under `system-prompts/`. I used 395 release tags, from v0.2.9 (published 24 February 2025) to v2.1.131 (published 6 May 2026) (npm, 2026): 275,889 (tag, path) pairs, 22,995 distinct file contents and 9,933 distinct file paths. A file counts as mentioning a word if the word appears, case-insensitively and on word boundaries, in the latest non-empty version of the file; the patterns are in section A. The mirror adds a generated summary to the top of each file, and I quote only the prompt text below it. It also stores some prompts as skeletons in which strings it could not recover appear as placeholders such as `${EXPR_1}`; where a quoted passage has a placeholder that matters, I took the literal from the Piebald-AI extraction (Piebald-AI, 2026).

In September 2026 I re-ran the counts on a fresh clone of the mirror. Every corpus-level count in this section reproduces the May 2026 figures exactly when one tag, v2.1.49, is left out; the mirror now carries that tag. With it the corpus has 396 tags and 10,032 paths, and no percentage below moves by more than 0.3 points. The per-file release counts used here also reproduce, except for two verify-change prompts, for which I give the recounted values.

4.2 A WORD COUNT AND WHAT IT MEASURES

Of the 9,933 files, 1,759 mention “user” in their latest version. Table 2 counts how many of those also mention each of 19 terms for programmatic entities, and “human”. The most common term, system, is ambiguous (file system, system prompt, system role), so I set it aside. At least one of the other 18 terms appears in 1,278 of the 1,759 files (72.7%); “human” appears in 74 (4.2%). The ratio of the two counts is 17 to 1.

That number describes how Claude Code’s prompts are written: their authors mostly mention the user in text that also involves tools, agents, hooks and APIs. It says nothing about what a model represents or does. It also sets a union of 18 terms against a single word, so its size depends on the term list: tool alone gives 13.6 to 1, and leaving out machine, service, service_account and AI as well as system still gives 1,245 files (70.8%, 16.8 to 1). Without a control corpus I cannot say whether technical writing in general looks the same (section 7). Of the 74 files that mention both “user” and “human”, 73 also mention a programmatic term.

Table 2. Files whose latest version mentions “user” and the given term, out of the 1,759 that mention “user”. *system* is ambiguous and is left out of the union in the last rows.

TERM	FILES	%	TERM	FILES	%
system	1,597	90.8	service	75	4.3
tool	1,010	57.4	LLM	48	2.7
agent	436	24.8	machine	46	2.6
API	372	21.1	teammate	43	2.4
hook	320	18.2	automated	43	2.4
AI	167	9.5	caller	41	2.3
subagent	145	8.2	programmatic	31	1.8
autonomous	88	5.0	plugin	31	1.8
script	87	4.9	bot	6	0.3
			service_account	2	0.1
any of the 18 terms other than system				1,278	72.7
human				74	4.2

The one that does not, `system-prompt-d7af86b4.md`, uses “human” only in “human-readable cadence”.

4.3 WHO THE PROMPTS SAY THE USER IS

The count is the weaker part of the corpus evidence. The stronger part is what individual prompts say, and the clearest statement is in the prompts that tell Claude Code how to verify a change (table 3). The first version, present in releases v2.1.83 to v2.1.87, says “The surface is where a user — human or programmatic — meets the code” and then tabulates who the user is at each surface: “a human at a terminal”, “an HTTP client”, “eyeballs on pixels”, “code that imports it”. From v2.1.88 (30 March 2026) on, every version (four system-prompt files and one skill file, through v2.1.118) says “meets the change”, drops the “User is” column, and lists two more surfaces that involve no person: “Prompt / agent config”, where the surface is “the agent”, and “CI workflow”, where it is “Actions”.

In these prompts “user” names a slot that a person or a program can fill, which is the distinction this paper draws. The prompts keep the noun and move “human” and “programmatic” into an aside between dashes.

Across the corpus, prompts place the user relative to programmatic entities in a few recurring ways (table 4). The relation is set by the surrounding prose: the same prompt can put the user above a tool when the tool serves the user’s request and beside it when error output is routed. Two of the patterns put non-human text into the user role directly. Teammate agents’ messages “appear automatically as new conversation turns (like user messages)”. Tool output does too. The Claude API reference that ships inside Claude Code (`data-claude-api-reference-curl.md`, v2.1.116 to v2.1.118) sends a tool result back to the model in a message with `"role": "user"` whose content is a block of `"type": "tool_result"`, and the summarization prompts ask the

Table 3. *The surface tables of the verify-change prompts, verbatim. Upper panel: system-prompt-verify-change-behavior.md, v2.1.83 to v2.1.87 (“DONE is” and “Example” columns omitted). Lower panel: the four later system-prompt files and skill-verify-skill.md, v2.1.88 to v2.1.118; literals from the Piebald-AI extraction, example links and the import example omitted.*

“The surface is where a user — human or programmatic — meets the code.”		
SURFACE	USER IS	
CLI / TUI	a human at a terminal	
Server / API	an HTTP client	
Desktop / browser GUI	eyeballs on pixels	
Library	code that imports it	
“The surface is where a user — human or programmatic — meets the change.”		
CHANGE REACHES	SURFACE	YOU
CLI / TUI	terminal	type the command, capture the pane
Server / API	socket	send the request, capture the response
GUI	pixels	drive it under xvfb/Playwright, screenshot
Library	package boundary	sample code through the public export
Prompt / agent config	the agent	run the agent, capture its behavior
CI workflow	Actions	dispatch it, read the run

model to “List ALL user messages that are not tool results” (18 files, from v1.0.14 in June 2025 to v2.1.131).

4.4 WHERE THE PROMPTS WRITE “HUMAN”

When a prompt needs a person rather than whatever fills the user slot, it changes the word. I sorted the 75 files in my catalog, the 74 that mention both “user” and “human” plus tool-description-memorable-moment-json.md, by their main use of “human” (table 5; the full list is in section C). The classification is mine, and a file can show more than one use.

Two of the uses carry the argument. D is where the prompts name the split outright. Besides the verify prompts it includes a managed-agents onboarding table whose Trigger column lists Webhook, Cron and Human as values of one field. A is where a prompt needs a person and says so. The Learn-by-Doing prompts ask the model to “Request human input on [specific decision]”; the skill-extraction prompts mark steps “[human] (user does it)” and ask for a “Human checkpoint” before irreversible actions; the autonomous-loop prompt aims for a pull request “ready to merge pending only human review”. In these prompts “user” would not be enough, because an agent or an API caller can fill the user slot and cannot do what the step asks.

The catalog covers 75 files. My September recount finds 107 files whose latest version contains “human”: the 74 that also mention “user”, and 33 that never mention “user” in any version.

Table 4. *How prompts relate the user to programmatic entities, with one verbatim example each. Tags: the number of releases whose extraction contains the file.*

PATTERN	EXAMPLE	FILE (TAGS)
one slot, two kinds	“The surface is where a user — human or programmatic — meets the change.”	system-prompt-verify-change-behavior-5.md (17)
arrives as user input	“These messages appear automatically as new conversation turns (like user messages)”	tool-description-create-and-manage-teams.md (33)
tool output in the user role	“All user messages: List ALL user messages that are not tool results.”	agent-prompt-conversation-summarization.md (13)
peer audiences	“Exit code \${NUM} - show stderr to subagent and continue having it run” and “Other exit codes - show stderr to user only”	system-prompt-handle-exit-codes-and-output.md (176)
user above the agent	“Your expertise lies in translating user requirements into precisely-tuned agent specifications”	system-prompt-architect-configs-from-needs.md (233)
user behind a relay	“the caller will relay this to the user, so it only needs the essentials”	system-prompt-anthropic-official-cli.md (50)

Besides the memorable-moment file, those 33 are 30 copies of a cron confirmation template (“the human-readable cadence”), one “human output” and one “like a security-conscious human would”. They would add to B and C and change nothing in D or A.

Table 5. *Uses of “human” in the 75 catalogued files, by main use.*

CODE	USE	FILES	EXAMPLE
D	names two kinds of user	7	“a user — human or programmatic”; a trigger table with the values Webhook, Cron and Human
A	a step needs a person	21	“Request human input on [specific decision]”; “[human] (user does it)”
C	a person as the benchmark	3	“like a security-conscious human would”
B	human-readable output	30	“the human-readable cadence”
T	term of art	13	“human-in-the-loop approval”
Q	tone of output	1	“Something human, funny, or surprising.”

4.5 RULES ENFORCED IN PROSE

Three prompts show what authors do when a rule depends on the character of a turn and the protocol cannot express it.

The security monitor for autonomous coding agents (four files, v2.1.71 to v2.1.119) tells the monitoring model: “Function names like `execute()` or `human_in_the_loop()` are programmatic wrappers — they are NOT human approval gates.” The rule is right, and it is enforced by a model reading a list of names. A wrapper with another name, such as `await_human_signoff()`, is not on the list. The wording also suggests that the authors had seen, or expected, a model treat such a call as approval; that is my inference from the text, not something the corpus records. A rule of the form “a person must approve this” is the kind of policy Palumbo et al. (2026) move out of the prompt and into a reference monitor.

The compaction prompt (`system-prompt-user-summary-file.md`, v2.1.86 to v2.1.131) asks the model that writes the summary to quote the person: “If there is a next step, include direct quotes from the most recent conversation showing exactly what task you were working on and where you left off. This should be verbatim to ensure there’s no drift in task interpretation.” The model that rewrites the conversation is the only check that its quotes are verbatim and that the framing around them is faithful, and the summary is what the work continues from. The same prompt restricts tool use in prose: “CRITICAL: Respond with TEXT ONLY. Do NOT call any tools.” and “Tool calls will be REJECTED and will waste your only turn — you will fail the task.”

The tool that schedules the next turn of an autonomous loop handles a turn with no person behind it: “For an autonomous `${PATH}` (no user prompt), pass the literal sentinel ``${EXPR_1}`` as ``prompt`` instead — the runtime resolves it back to the autonomous-loop instructions at fire time.” (`tool-description-schedule-resume-work-dynamic-mode.md`, v2.1.116 to v2.1.131). Here the character is typed, as a sentinel value in a tool call, and the runtime then replaces the sentinel with instructions that reach the model in the prompt’s place.

4.6 HOW STABLE THE WORDING IS

The wording is stable. In none of the 74 files did a later version drop “human”, and in 73 it was present from the file’s first version; the exception, a status-line prompt, gained “Human-readable session name” in v2.1.41. What did change in 16 files is how many times they say “human” or “user”. The Learn-by-Doing wording is the oldest. It entered the system prompt in v1.0.63 (29 July 2025), as `system-prompt-learn-by-doing-human-contributions.md`, which was renamed `system-prompt-learn-by-doing-human-input.md` at v1.0.86. It entered the tool description in v1.0.78, which kept it for 216 releases, to v2.1.119 (23 April 2026). When Claude Code renamed a file, the “human” wording moved with it: the background-job prompt of v2.1.117 became `agent-prompt-background-job-agent-instructions.md` by v2.1.128 with its three uses of “human” intact (Piebald-AI extraction).

Anyone re-checking these files at a single tag should know that the mirror changed its extraction at v2.1.119. The number of files under `system-prompts/` fell from 633 at v2.1.118 to 214 at v2.1.119 and 82 at v2.1.131, while the composite prompt `cc-prompt.md` stayed about the

same length (916, 916 and 912 lines). A search of v2.1.131 alone finds few of the files cited here; the corpus is the union over all 395 tags.

5 TESTING WHETHER MODELS TRACK WHO IS SPEAKING

The evidence so far is about protocols and prompts. Whether the gap changes model behavior depends on what the model represents, and I have not measured that. This section states the conjecture and how to test it.

Table 6. *Predictions and the results that would refute them.*

PREDICTION	TEST	REFUTED IF
R. Runtime text is placed by its surface form.	Apply role probes in the manner of Ye et al. (2026) to typed user turns, runtime-written user-role items (environment blocks, AGENTS.md, interruption notices, compaction summaries), tool outputs and quoted untrusted text. Expected: tag-wrapped runtime blocks sit with tool output, prose-like ones with typed user turns.	runtime-written items form a separable class of their own, whatever their form.
H1. A “typed by a person” direction exists but is unstable.	Extract the direction from contrastive pairs that vary the character cues and hold the content fixed; compare its stability across paraphrases with persona-vector traits (Chen et al., 2025) and the refusal direction (Arditi et al., 2024).	it is as stable as those directions.
H2. Steering it does little.	Ablate or add the direction and measure behavior that should depend on character: refusal calibration, adapting to expertise, asking for human approval. Compare with persona and refusal directions of similar norm.	its effects are as large as theirs.
H3. Behavior follows the cue more than the content.	Hold the task fixed and vary the cue (“a user ...”, “a human ...”, “an automated agent ...”, no cue); separately, change the content and hold the cue fixed. Compare the variance in refusal rate, adaptation and gating.	content changes move behavior more than cue changes.

Two things work against a model representing the character of a user-role turn. At inference time the protocol gives no signal: runtime text and typed text arrive in the same role, and the only cues are in the text. In training, if agentic transcripts look like the sessions in section 3, the user role is often filled by something other than a person. I do not know how much text of this kind current models are trained on, so the second point is a conjecture. Ye et al. (2026) measured what happens to injected text: models place it in whatever role it sounds like. Runtime text in the user role raises the same question with no attacker involved.

Table 6 states four predictions. Each can be run on open models with probes, sparse-autoencoder features and steering. The Codex logs supply labels for free: every user-role item

in a rollout log is marked by the presence or absence of a `user_message` event, so a probe can be trained and scored on real runtime traffic rather than on synthetic prompts.

If R and H3 come out as predicted, the model infers the character from surface form, which is all the protocol leaves it, and a typed field would give it something better. If the probes find a separable class for runtime text, models already recover the character better than the protocol suggests, and the case for the field rests on enforcement alone. Probes and features read activations directly, so a positive result on H1 would be a fact about the forward pass, not about what the model writes in its reasoning.

Two checks on the corpus side would bound the word count of [section 4.2](#), though neither tests a model. A control corpus would run the same count on public Codex documentation, general technical Markdown sampled by file length, and a random sample of GitHub Markdown. A windowed count, within 50 tokens or within a paragraph, would replace file-level co-occurrence, since “user” and “agent” can sit in different parts of a long prompt.

6 A TYPED CHARACTER FIELD

Both systems know the character of a user-role turn somewhere, and both deliver it to the model, if at all, as text. Codex records it in runtime state: the `user_message` event marks typed input, and the remote compaction path uses an item type of its own. What reaches the model as readable text is XML-like tags around runtime blocks and, in the local compaction path, a fixed prefix on summaries. Claude Code’s prompts record it in prose: “a user — human or programmatic”, “[human] (user does it)”, “Human checkpoint”, “NOT human approval gates”, “user messages that are not tool results”. Its scheduling tool comes closest to a typed value, with a sentinel that the runtime resolves. Claude Code does type neighboring things: at v2.1.131 its memory prompt defines four named memory types (user, feedback, project, reference), and teammates are addressed by name with `to: "<name>"`, or `to: "*" for a broadcast`. None of these types the source of a user-role turn. This is my reading of two systems. I did not observe either vendor’s intentions, and I do not claim that they reached the same diagnosis independently.

6.1 THE PROPOSAL

Each message should carry a character field beside its role, set by the runtime and not by the message text. The values can start from what runtimes already distinguish: typed by a person, runtime context (environment blocks, instruction files, skill bodies), runtime notices such as interruptions, compaction summaries, messages from other agents, tool output, and scheduled triggers. The role stays as it is; models rely on it, and nothing here argues that it is broken.

A model can only use the field if it sees the field in training, consistently. [Chen et al. \(2025\)](#) extract persona vectors from contrasting examples of a trait; the analogous data for the user side is cheap to produce, because a runtime already knows the character of every item it sends. Rules that depend on the character, such as “a person must approve this”, belong in the runtime and in a monitor that enforces them ([Palumbo et al., 2026](#)). The field lets the model reason about who is speaking; the monitor makes the rule hold whether or not it does.

6.2 A CAUTION ABOUT COARSE TYPES

In a companion paper (Fitch, 2026) I read the 163 commits between `rust-v0.117.0-alpha.6` and `rust-v0.117.0-alpha.24` (20 to 26 March 2026). Several of them replaced string identifiers with typed structures: a `ProcessId` type for processes and typed payloads for tool-use hooks. `AgentMetadata`, which already carried `agent_path` and `agent_nickname` for agents, moved into the agent registry in the same range. None of these types the source of a user-role message, and in the March 22 session, under `0.117.0-alpha.8`, user-role items still reached the model with no such field. The same paper argues that a typed carrier built on a poor default makes the default harder to change, because more code comes to depend on it. For the character field, a single human-or-programmatic bit would be such a default: it would merge a compaction summary, a teammate’s message and a scheduler’s trigger into one value. The uses of “human” in section 4.4 and the item types in section 3 are a first list of what the field has to express, and the list itself is a research question.

7 LIMITATIONS

Each layer rests on one system: the runtime evidence on Codex CLI, the prompt evidence on Claude Code. That supports a claim about these two systems and not about agent runtimes in general; Gemini CLI, Aider and Cursor would be the next places to look.

The Codex evidence is four sessions, all mine, run on my own modified builds. Three share a prompt, and two of those share a configuration. The prologue and the event asymmetry hold in every session I have, over 60 user-role items, but I have not checked stock builds or other versions. The summary-as-user-message path is documented from source code and was not observed in these logs.

The prompts come from a third-party extraction. Placeholders can hide a “user”, “human” or programmatic term, so a count can be off in either direction, and I checked the main quoted passages against a second extraction, not every file. The word count has no control corpus and works at the level of whole files. The classification of the uses of “human” is mine and has not been replicated.

I do not know whether prompts like these enter training, or how a model would weigh them if they did, and the predictions in section 5 are untested. If H1 fails, the argument that the user side is less well represented than the assistant side weakens; the protocol evidence stands either way.

The agreement among the three March 21 runs follows from their shared input (section 3.6), so it illustrates the argument and tests nothing.

The typed-carrier observation and the caution about coarse types come from my own companion paper, not from an independent source. I re-checked its specific claims used here in the Codex source: `ProcessId` (`exec-server/src/process_id.rs`) and the hook payload structures (`core/src/tools/registry.rs`) are absent at `rust-v0.117.0-alpha.6` and present at `rust-v0.117.0-alpha.24`, and `AgentMetadata` with its `agent_path` and `agent_nickname` fields is present at both.

Last, “character” is borrowed from the persona-vector work as a lens, not derived from it. If persona-vector methods depend on conditions that do not carry over to the user side, the predictions have to be restated.

DATA AND TOOLS

The Claude Code corpus is public: the mirror and tag range are given above, and the patterns and counts are in section A. The Codex rollout logs and terminal captures are on file with me. They contain local paths and a private repository’s instructions, so they are described rather than published: section B gives file names, sizes, session identifiers and the item numbers cited, so that each claim can be checked against them.

The four Codex sessions ran on my own modified builds of the open-source Codex CLI (section 3.1). This revision was drafted with Claude Code, Anthropic’s coding agent, running Claude Opus 5.5, from my May 2026 version, and the corpus counts in section A were recomputed during the revision with Python scripts. Claude Code is also one of the two systems this paper examines. For that reason every claim about it rests on its public prompt corpus, which can be checked without it.

REFERENCES

- A. Ardit, O. Obeso, A. Syed, D. Paleka, N. Panickssery, W. Gurnee, and N. Nanda. Refusal in language models is mediated by a single direction. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. arXiv:2406.11717.
- R. Chen, A. Ardit, H. Sleight, O. Evans, and J. Lindsey. Persona vectors: Monitoring and controlling character traits in language models. arXiv:2507.21509, 2025.
- J. Z. Fitch. Inherit All of Nothing. 2026. <https://johnzfitch.github.io/johnzfitch/papers/inherit-all-of-nothing.pdf>.
- marckrenn. claude-code-changelog: versioned extraction of Claude Code system prompts. GitHub repository, tags v0.2.9 to v2.1.131, cloned September 2026. <https://github.com/marckrenn/claude-code-changelog>.
- npm registry. Package @anthropic-ai/claude-code, version publication times. <https://www.npmjs.com/package/@anthropic-ai/claude-code>, accessed September 2026.
- OpenAI. Model Spec, version of 11 April 2025. <https://model-spec.openai.com/2025-04-11.html>.
- OpenAI. Codex CLI source code, tag rust-v0.116.0-alpha.11, file codex-rs/core/src/compact.rs. <https://github.com/openai/codex>.
- N. Palumbo, S. Choudhary, J. Choi, P. Chalasani, M. Christodorescu, and S. Jha. Policy compiler for secure agentic systems. arXiv:2602.16708v1, 2026. Version 2 drops M. Christodorescu

from the author list. Version 3 adds G. Amir and is titled *Formal policy enforcement for real-world agentic systems*.

Piebald-AI. claude-code-system-prompts. GitHub repository, releases since v2.0.14, cloned September 2026. <https://github.com/Piebald-AI/claude-code-system-prompts>.

A. Templeton, T. Conerly, J. Marcus, J. Lindsey, T. Bricken, B. Chen, A. Pearce, C. Citro, E. Ameisen, A. Jones, H. Cunningham, N. L. Turner, C. McDougall, M. MacDiarmid, A. Tamkin, E. Durmus, T. Hume, F. Mosconi, C. D. Freeman, T. R. Sumers, E. Rees, J. Batson, A. Jermyn, S. Carter, C. Olah, and T. Henighan. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024. <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html>.

E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel. The instruction hierarchy: Training LLMs to prioritize privileged instructions. arXiv:2404.13208, 2024.

C. Ye, J. Cui, and D. Hadfield-Menell. Prompt injection as role confusion. arXiv:2603.12277, 2026. ICML 2026.

J. Zhang, T. Li, W. Jurayj, H. Zhan, B. Van Durme, and D. Khashabi. Many-tier instruction hierarchy in LLM agents. arXiv:2604.09443, 2026. Findings of EMNLP 2026.

A CORPUS METHOD AND COUNTS

The corpus is every file under system-prompts/ at each of the 395 release tags of the marckrenn mirror from v0.2.9 to v2.1.131, leaving out v2.1.49. Each (tag, blob, path) triple is one record. For each path, the *latest version* is its content at the last tag where the content is not empty. All patterns are case-insensitive regular expressions, each wrapped in word boundaries (\b):

PATTERN	USED FOR
users?, humans?	“user” and “human”
TERMS?	system, tool, agent, API, hook, AI, script, service, LLM, machine, teammate, caller, plugin, bot
sub-?agents?	subagent
autonomous(ly)?	autonomous
automat(ed ion)	automated
programmatic(ally)?	programmatic
service[_ -]accounts?	service_account

These patterns reproduce every row of table 2 exactly on the September 2026 clone. Co-occurrence is counted per file, not per paragraph. The “tags” count given for a file (table 4 and section C) is the number of release tags whose extraction contains that path.

The ratio in section 4.2 is 1,278 to 74. The 50 paths counted in 1,809 but not in 1,759 are of two kinds. In 33 of them the latest version mentions “human” but not “user”, and no version

Table 7. *The corpus counts and the scope each is computed over.*

COUNT	WHAT IS COUNTED	SCOPE
275,889	(tag, blob, path) records	all 395 tags
22,995	distinct file contents (blobs)	all 395 tags
9,933	distinct file paths	all 395 tags
1,809	paths with “user” or “human” in some version	all versions
1,759	paths with “user”	latest version
1,278	paths with “user” and at least one of the 18 terms other than system	latest version
107	paths with “human”	latest version
74	paths with both “user” and “human”	latest version
33	paths with “human” and never “user”	every version
1	paths with “user” and “human” and none of the 18 terms (system-prompt-d7af86b4.md)	latest version

mentions “user”. In the other 17 the latest version mentions neither word: 16 mentioned “user” in an earlier version and one mentioned “human”. With tag v2.1.49 included the corpus has 396 tags and 10,032 paths; 1,771 paths mention “user”, 1,283 of them (72.4%) also mention one of the 18 terms and 74 (4.2%) mention “human”.

B CODEX ARTIFACTS

Item numbers are 0-based line numbers in a JSON Lines file, which the Codex CLI calls a rollout; line numbers in the terminal captures are 1-based. I read the logs with a JSON parser, one line at a time.

Table 8. *The Codex files. Sessions are identified by the identifier the CLI records in the first line of each rollout.*

FILE	SIZE	NOTES
rollout-exec.jsonl	33 items	codex exec; session 019d0fba-e258-7ae0-b429-41e831577d48; prologue items 1 to 6; final answer item 29
exec.txt	461 lines	its capture: an interrupted attempt and the rerun, the whole session appearing twice; the prompt header at lines 22, 69, 245 and 292; the echoed command runs into a log line at 81–82 and 304–305
rollout-exec-lite.jsonl	84 items	codex exec --lite; session 019d0fc0-654e-7793-a005-45b450932210; reasoning item 77; final answer item 80
exec-lite.txt	516 lines	its capture

continued on the next page

FILE	SIZE	NOTES
rollout-exec-lite-danger.jsonl	51 items	codex exec --lite-danger; session 019d0fb1-d1ee-71a0-ae0c-9175f17790fb; final answer item 47
exec-lite-danger.txt	949 lines	its capture; the reasoning and final answer appear three times (final answer at lines 302, 609 and 934)
rollout-2026-03-22T07-33-57-019d15f7-8a1a-7d21-a2fd-ec1106e13cde.jsonl	1,070 items	interactive CLI, 0.117.0-alpha.8; AGENTS.md at item 3; typed request at item 5; <turn_aborted> at item 931
rollout-2026-03-21T00-37-55-019d0f54-4991-7793-a686-e8e4498f0e0f.jsonl	1,412 items	the earlier session the March 21 runs read (0.116.0-alpha.11); compactions at items 657 and 1101; pasted model text in the user_message at item 1106
codex-full-chain.txt		the terminal capture of that session, the runs' other input

C THE FILES THAT USE “HUMAN”

The catalogued uses, grouped by the wording they share. File identifiers refer to [table 9](#); every phrase below appears verbatim in the latest version of every file listed with it.

D, names two kinds of user. “The surface is where a user — human or programmatic — meets the change.” (D-1, D-2, D-5, D-6, D-7); “... meets the code.” and “FAIL with the raw output attached so a human reads it instead of you guessing” (D-4); the onboarding table rows “Event-triggered | Webhook”, “Scheduled | Cron”, “Fire-and-forget PR | Human” and “Research + dashboard | Human” (D-3).

A, a step needs a person. “Request human input on [specific decision]” (A-1, A-2, A-5, A-16, A-19, A-20); “[human] (user does it)” and “Human checkpoint” (A-3, A-10, A-17); “permissions similar to a human developer” (A-4, A-8, A-12, A-14); “without human approval or established safety frameworks” (A-6, A-9, A-11, A-21); “ready to merge pending only human review” (A-7); “reduce the number of human interactions required from users” (A-13, A-15); “If the human replies, open your next turn by restating what they said” (A-18).

C, a person as the benchmark. “like a security-conscious human would” (C-1, C-3); “For keys tied to a human” (C-2).

B, human-readable output. “the human-readable cadence”, in a cron confirmation template (B-4, B-9, B-10, B-13, B-14, B-16, B-17, B-21, B-22, B-23, B-24, B-28); “Human-readable name (always use this for messaging and task assignment)”, in the team tools (B-2, B-8, B-12, B-15, B-18, B-25, B-26); “Human-readable session name” (B-1, B-3, B-5, B-27); “schedule (human-readable)” (B-6, B-7, B-29, B-30); “Human-readable title” (B-11); “convert cron to human-readable” (B-19); “anything a human maintains” (B-20).

T, term of art. “human-in-the-loop approval”, in SDK tutorial text (T-1 to T-13).

Q, tone of output. “Something human, funny, or surprising.” This file never mentions “user”: tool-description-memorable-moment-json.md, 74 tags.

Table 9. The 74 files whose latest version mentions both “user” and “human”, by number of release tags. Uses: main use first, codes as in table 5.

ID	FILE	TAGS	USES
A-1	tool-description-collaborative-learning-cli.md	216	A
A-2	system-prompt-learn-by-doing-human-input.md	208	A
A-3	system-prompt-extract-repeatable-session-steps.md	62	A
B-1	agent-prompt-shell-ps1-to-statusline.md	34	B
T-1	system-prompt-typescript-use-overview.md	34	T
T-2	system-prompt-use-python-overview.md	34	T
B-2	tool-description-create-and-manage-teams.md	33	B
B-3	agent-prompt-status-line-setup.md	32	B
B-4	system-prompt-schedule-recurring-with-cron.md	31	B
B-5	agent-prompt-ps1-to-statusline.md	29	B
B-6	system-prompt-schedule-remote-agents-3.md	27	B
B-7	system-prompt-schedule-remote-agents-4.md	27	B
B-8	tool-description-spawn-and-manage-teams.md	26	B
B-9	system-prompt-schedule-recurring-2.md	24	B
B-10	system-prompt-schedule-recurring-interval.md	24	B
A-4	system-prompt-monitor-autonomous-coding.md	23	A, C
A-5	system-prompt-learn-by-doing-human-contributions.md	22	A
T-3	system-prompt-use-concepts-api-2.md	21	T
A-6	system-prompt-trusted-repositories-data-handling.md	18	A
T-4	system-prompt-python-runner-beta.md	18	T
B-11	system-prompt-managed-agents-core-concepts.md	17	B
D-1	system-prompt-verify-change-behavior-5.md	17	D, C
A-7	system-prompt-autonomous-loop-check.md	15	A
D-3	system-prompt-managed-agents-onboarding-flow.md	14	D
T-5	system-prompt-concepts-this-file-covers-conceptual.md	13	T
T-6	system-prompt-type-script-conceptual-overview-definitions.md	13	T
T-7	system-prompt-use-concepts-api.md	13	T
A-8	system-prompt-monitoring-autonomous-coding-agents.md	12	A, C
B-12	tool-description-manage-teams-for-collaboration.md	12	B
A-9	system-prompt-trusted-repo-external-services.md	11	A
B-13	system-prompt-cron-expression-confirmation.md	11	B
D-2	system-prompt-verify-change-behavior-2.md	11	D, C
T-8	system-prompt-use-concepts.md	11	T
A-10	system-prompt-session-analysis-reusable-skills.md	10	A
B-14	system-prompt-memory-consolidation-team-reflection.md	9	B
B-15	tool-description-manage-swarm-teams.md	9	B
T-9	system-prompt-python-conceptual-overview-definitions-choice.md	8	T
C-1	system-prompt-submit-plan-for-approval.md	7	C
A-11	system-prompt-environment-user-replace-trusted-repo.md	6	A
A-12	system-prompt-security-monitor-autonomous-ai-coding.md	6	A, C

continued on the next page

ID	FILE	TAGS	USES
B-16	system-prompt-d7af86b4.md	6	B
C-2	system-prompt-managed-agents-client-patterns.md	5	C
D-4	system-prompt-verify-change-behavior.md	5	D
A-13	system-prompt-opus-thinking-tokens.md	4	A
A-14	agent-prompt-security-monitor-for-autonomous-agent-actions-first-part.md	4	A, C
B-17	system-data-cron-expressions-custom-rules.md	4	B
B-18	tool-description-manage-teams-and-approve-plans.md	4	B
D-5	system-prompt-verify-change-behavior-4.md	4	D, C
A-15	skill-model-migration-guide.md	3	A
A-16	system-prompt-learning-mode.md	3	A
A-17	system-prompt-skillify-current-session.md	3	A
A-18	system-prompt-background-job-behavior.md	3	A
B-19	agent-prompt-schedule-slash-command.md	3	B
B-20	data-managed-agents-core-concepts.md	3	B
B-21	skill-loop-slash-command.md	3	B
B-22	skill-schedule-recurring-cron-and-execute-immediately-compact.md	3	B
D-6	skill-verify-skill.md	3	D, C
T-10	data-tool-use-concepts.md	3	T
T-11	data-tool-use-reference-python.md	3	T
T-12	data-tool-use-reference-typescript.md	3	T
A-19	system-prompt-interactive-cli-helps-users-software.md	2	A
A-20	tool-description-pauses-asks-write-small-pieces.md	2	A
A-21	system-prompt-trusted-repo-external-services-2.md	2	A
B-23	system-prompt-cron-expression-github-sync.md	2	B
B-24	system-prompt-github-cron-customization.md	2	B
B-25	tool-description-teammate-tool.md	2	B
B-26	tool-description-manage-teams-and-teammates.md	2	B
B-27	agent-prompt-this-configure-user-status-line.md	2	B
T-13	system-prompt-use-concepts-schema.md	2	T
B-28	system-data-custom-rules-github-cron.md	1	B
B-29	system-prompt-schedule-remote-agents-2.md	1	B
B-30	system-prompt-schedule-remote-agents.md	1	B
C-3	tool-description-submit-plan-for-approval.md	1	C
D-7	system-prompt-verify-change-behavior-3.md	1	D, C